

PyCJ Simple Book

Made By Arshman Anil

A simple, professional guide to learning
computer programming.

Perfect for beginners taking their first steps into
coding.



Beta Version



Welcome to PyCJ! Please note that this programming language is currently in its BETA testing phase.

Our Mission

"To help beginners master the fundamentals of logic, so they can easily understand any other programming language in the future."

Table of Contents

Step 1: What is a Variable?

Step 2: Saving Words (Strings)

Step 3: Saving Numbers

Step 4: True or False Flags

Step 5: Showing Output on Screen

Step 6: Mixing Words and Variables

Step 7: The 'If' Choice Check

Step 8: The 'Else' Backup Plan

Step 9: The Repeat Loop

Step 10: The For Loop

Step 1: What is a Variable?

A variable is like a digital storage box. You can put a piece of information inside this box and give it a name. By giving it a short, clear name, the computer will remember it for you.

```
imagine myBox = 5
```



Definition

We use the special keyword **imagine** to tell the computer to create a brand new box in its memory.



Try This

Write a line of code to make a box named **myScore** and put the number **100** inside it.

Step 2: Saving Words

When you want to save letters or words, you must put them inside quotation marks. The quotation marks tell the computer that it is looking at plain text, not a programming command.

```
imagine myName = "Arshman Anil"
```



Definition

In the programming world, words and sentences placed inside quotation marks are officially called **Strings**.



Try This

Create a variable named **myFood** and save the string **"Pizza"** inside it.

Step 3: Saving Numbers

Unlike words, numbers do not need quotation marks. You can save standard whole numbers, or you can use a decimal point to save exact measurements.

```
imagine myCoins = 10  
imagine myHeight = 5.8
```



Pro Tip

Because there are no quotes around them, the computer knows these are numbers. This means you can add, subtract, and multiply them later!



Try This

Make a box named **myAge** and put your current age inside it without using quotes.

Step 4: True or False Flags

Sometimes you only need to know if something is "On" or "Off". In PyCJ, we use the word **true** to mean "On/Yes", and **false** to mean "Off/No".

```
imagine isGameOn = true  
imagine isGameOver = false
```



Definition

These true/false variables are called **Booleans**. They help the computer make very fast decisions.



Try This

Create a variable named **isHappy** and set its value to **true**.

Step 5: Showing Output

A program needs to talk to the user. To see what is hidden inside your variables, or to print a message, you use the **output** command. This will show the result directly on the screen.

```
output("System is starting...")  
output(myCoins)
```

Pro Tip

If you put a variable's name inside the output parentheses, the computer opens the box and prints the value inside it.

Try This

Write the code needed to display the message "**Good Job!**" on the screen.

Step 6: Mixing Words and Variables

Sometimes you want to print a sentence that includes a variable. By typing the letter **p** right before your quotation marks, you unlock the ability to inject variables using curly brackets `{ }`.

```
output(p>Welcome back, {myName}!")
```



Definition

This feature is called **Interpolation**. The computer automatically replaces the curly brackets with your saved data.



Try This

Print a message that says **"I have {myCoins} coins"** using the 'p' string method.

Step 7: The 'If' Choice Check

Programs need to make decisions. An **if** statement checks a condition. If that condition is true, the computer runs the secret code hidden inside the curly brackets.

```
if myCoins > 5 {  
    output("You have enough coins to  
play!")  
}
```

Pro Tip

If the check fails (for example, if you only have 2 coins), the computer simply skips the brackets and does nothing.

Try This

Write an **if** check to see if the variable **myScore** is greater than 50.

Step 8: The 'Else' Backup Plan

What happens when your **if** condition fails? You can provide a safe backup plan using the **else** command. This ensures your program always knows what to do.

```
if myCoins > 5 {  
    output("You win!")  
} else {  
    output("You need more coins. Try  
again!")  
}
```



Definition

The **else** block does not need a condition to check. It is the default action that handles everything else.



Try This

Add an **else** block to your previous code that prints "**Game Over**" if the score is not high enough.

Step 9: The Repeat Loop

If you want the computer to do the same task over and over again, you use a loop. The **repeat** loop works exactly like an 'if' statement, but it keeps running its code until the condition becomes false.

```
imagine energy = 3

repeat energy > 0 {
  output("Running...")
  energy = energy - 1
}
```



Warning

Always remember to change the variable inside the loop (like subtracting energy). If the condition is always true, the loop will run forever and crash your program!



Try This

Write a repeat loop that prints "Jumping!" as long as a **stamina** variable is greater than 0.

Step 10: The For Loop

When you know exactly how many times you want a loop to run, the **for** loop is the best tool. It keeps all your counting rules neatly organized inside parentheses.

```
for (imagine i = 1, i <= 5, i = i + 1) {  
    output(p"This is loop number {i}")  
}
```



Definition

A for loop has 3 parts separated by commas: The **Start** (imagine $i = 1$), the **Rule** ($i \leq 5$), and the **Step** ($i = i + 1$).



Try This

Write a **for** loop that counts from 1 to 10 and outputs the number each time.



Congratulations!

You have completed the PyCJ Simple Book.

You now understand the core fundamentals of programming. You know how to store data, make decisions, and repeat actions.

Keep practicing, keep experimenting, and keep writing amazing code!

PyCJ - Created By Arshman Anil